Practical tutorial for `linstabpy`

This tutorial is for new users who want to run the code before changing it. It walks through the installation check, the Martini example family, the Song 2023 validation case, and the pattern for adapting a shipped case to a new configuration.

1. What you will do

At the end of this tutorial you will have:

- installed or checked the local `linstabpy` runtime,
- run a fast smoke example before starting expensive examples,
- run each Martini example from its paired Python and JSONC files,
- understand where reusable code, examples, studies, and validation assets live,
- use the single example runner for serial, parity, and MPI-safe runs,
- run the MATLAB-parity comparison for the Martini examples,
- run the Song 2023 baseline and its JSONC-driven validation matrices,
- learned how to copy a case and adjust parameters for a new study.

Note

Every blue box contains terminal commands to type from the repository root unless another directory is stated.

Warning

When you see `/path/to/linstabpy`, replace it with the real path to your local checkout. In this repository, use `.venv/bin/python`. Do not replace these commands with `uv run` for PETSc/SLEPc runs.

2. Repository and runtime check

Move to the repository root and inspect the environment before installing anything.

Command

```
cd /path/to/linstabpy
bash diagnose.sh
```

If the diagnostic reports missing system packages on Linux, install or load them outside this tutorial. The installer is written to avoid silent system changes by default.

Install the project runtime:

Command

```
bash install.sh
bash install.sh --check
```

The install script creates `.venv/` and installs PETSc/SLEPc support for the project. After installation, verify the Python used by the examples:

Command

```
.venv/bin/python --version
.venv/bin/python -c "import linstab2d; print(linstab2d.__file__)"
```

Note

The import package is `linstab2d`. The repository and distribution name are `linstabpy`. Keep this distinction in mind when reading commands and imports.

3. The code pattern

Most runnable studies in this repository follow a paired-file pattern:

Expected output

```
examples/example_1_blasius.py
examples/example_1_blasius.jsonc
```

The Python file contains the workflow. The JSONC file contains the run parameters. For first modifications, change the JSONC file or copy the pair. Avoid hardcoding study parameters inside the package code.

Note

You may not be familiar with JSONC. It is JSON with comments, and it is used here deliberately even though plain JSON is more common. These examples are numerical cases, so the case definition should be readable and reviewable: Reynolds number, Mach number, grid size, boundary conditions, sponge parameters, forcing masks, solver targets, and PETSc/MPI choices all belong in a visible input file. The comments explain why a parameter exists and whether it comes from the Martini reference case or from the Python port. Plain JSON would remove those explanations, command-line flags would hide too much state in long commands or shell history, and Python config files would mix data with executable code.

The examples also follow a cache pattern:

Command

```
.venv/bin/python -m examples.example_1_blasius
```

If the matching `.npz` already exists, the script usually loads it and plots. To recompute, remove the matching `.npz` and rerun:

Command

```
rm -f examples/example_1_blasius.npz
.venv/bin/python -m examples.example_1_blasius
```

Warning

The full examples can be memory- and time-intensive. Start with the smoke run and the cached outputs before deleting large `.npz` artifacts.

Where files belong

The top-level directories have distinct roles. Keep reusable package code, runnable examples, study campaigns, and validation data separated:

Expected output

```
src/linstab2d/      importable Python package
examples/           runnable Martini examples; generated .npz/.png ignored
studies/case1/      SWBLI study: configs, data/baseflow, reference, scripts
studies/case2/      TCFD campaign: configs, data/baseflow, scripts, docs
validation/         reproducibility harnesses, fixtures, reports, references
validation/martini2024/tools/
                    MATLAB-to-Python conversion and parity tooling
scripts/            repository guardrails and maintenance utilities
install_tester/     thin install/diagnose/benchmark harness
tutorial/           tutorial source and generated PDF
```

Do not put reusable solver code under `studies/` or `examples/`. Put reusable code under `src/linstab2d/`, runnable examples under `examples/`, case-specific workflows under `studies/`, and reference/validation assets under `validation/`. Generated outputs such as example `.npz`/`.png` files, study `output/` directories, caches, and build metadata are ignored and should not define the repository structure.

Use one example runner

`run_all_examples.sh` is the single documented runner. It covers serial example passes, cache cleanup, Martini parity, and MPI-safe launches. Direct module execution is still useful for local debugging, but use the runner for acceptance checks and MPI runs.

Command

```
bash run_all_examples.sh --list
bash run_all_examples.sh
bash run_all_examples.sh --fresh --example 1 --skip-parity
bash run_all_examples.sh --clean --example 1 --skip-parity
```

For MPI runs, enable `-mpi`. If `-n/-ranks` is omitted, the runner starts from half of the detected physical cores. The runner also sets one OpenMP/BLAS thread per rank unless those variables were already set by the caller.

Command

```
bash run_all_examples.sh --mpi -m examples.example_1_blasius_smoke
bash run_all_examples.sh --mpi -n 10 -m examples.example_3_streaky_bl
bash run_all_examples.sh --mpi -n 1 -m examples.example_1_blasius_smoke -- -eps_monitor
```

Note

The previous separate MPI wrapper has been folded into `run_all_examples.sh`. Override the MPI launcher or default rank count with `LINSTABPY_MPIEXEC` and `LINSTABPY_MPI_RANKS` when a machine needs explicit runtime paths.

4. First run: Blasius smoke test

The fastest first test uses the same physics path as the Blasius example but on a smaller grid.

Command

```
.venv/bin/python -m examples.example_1_blasius_smoke
```

Expected outputs:

Expected output

```
examples/example_1_blasius_smoke.npz
examples/example_1_blasius_smoke.png
```

The smoke JSONC file is:

Expected output

```
examples/example_1_blasius_smoke.jsonc
```

Use it to understand the basic blocks: flow parameters, grid size, sponge, forcing mask, boundary-condition recipe, assembly backend, and solver settings.

5. Martini example family

The Martini examples are the first complete tour through the library. They progress from a numerical derivative check to resolvent analysis, temporal stability, spatial stability, direct and adjoint modes, quasi-structured meshes, curvilinear coordinates, and axisymmetric operators. The physical descriptions below follow the Martini and Schmidt example text, while the commands and artifacts refer to this Python port.

Expected output

```
0  example_0_curvilinear_validation  derivative and curvilinear-grid check
1  example_1_blasius                 Blasius boundary-layer resolvent
2  example_2_jet                     axisymmetric jet resolvent
3  example_3_streaky_bl              temporal and spatial stability
```

```
4  example_4_cavity                 direct and adjoint cavity EVP
5  example_5_paraboloid             curvilinear axisymmetric resolvent
```

5.0 Common run and figure pattern

For acceptance, use the runner. For learning a single script, direct module execution is fine.

Command

```
bash run_all_examples.sh --list
bash run_all_examples.sh --fresh --example 2 --skip-parity
bash run_all_examples.sh --clean --example 2 --skip-parity
```

Each example writes an `.npz` cache and one or more `.png` figures in `examples/`. When the `.npz` exists, the script usually plots from cache. Remove the matching `.npz`, or use `-clean`, when you need to recompute the numerical solve.

Warning

The figures shown in this tutorial are generated artifacts. They are ignored by git, so a fresh checkout may show placeholder notes until the examples are run locally.

Read the figures in the same order in every case:

- Start with the baseflow figure. Confirm the domain, coordinate system, sponge region, and any mask or body geometry before looking at modes.
- Then inspect the spectrum or gain summary. For resolvent cases, large separation between leading and suboptimal gains means the leading forcing/response pair is easier to interpret. For stability cases, the sign of the growth coordinate matters.
- Use the mode/support figures for localization. Individual color limits show shape; shared color limits show which component or region carries most of the plotted amplitude.
- Treat residual and parity reports as numerical health checks. A polished figure is not a substitute for a small residual or a passing parity comparison.

If a plot looks suspicious, separate three issues before changing code: numerical convergence, plotting choices, and physical interpretation. For example, a misplaced colorbar is a figure-quality issue; a mode selected from the wrong part of the spectrum is a numerical-study issue; a wavepacket located in the wrong shear layer is a physics/parity issue.

5.1 Example 0: curvilinear derivative validation

Example 0 is the numerical preflight. It has no flow physics. It builds a symmetric computational mesh, maps it to a parabolic physical domain, evaluates a known function, and checks whether the curvilinear derivative operator differentiates it correctly. Use it before interpreting physical modes, because it exercises the mapping and finite-difference machinery without solver ambiguity.

Command

```
.venv/bin/python -m examples.example_0_curvilinear_validation
bash run_all_examples.sh --fresh --example 0 --skip-parity
```

Main knobs:

File excerpt

```
"nx": 50,
"ny": 50,
"fd_order": 4,
"alpha_filter": 0.0,
"conformal_scale": 0.2
```

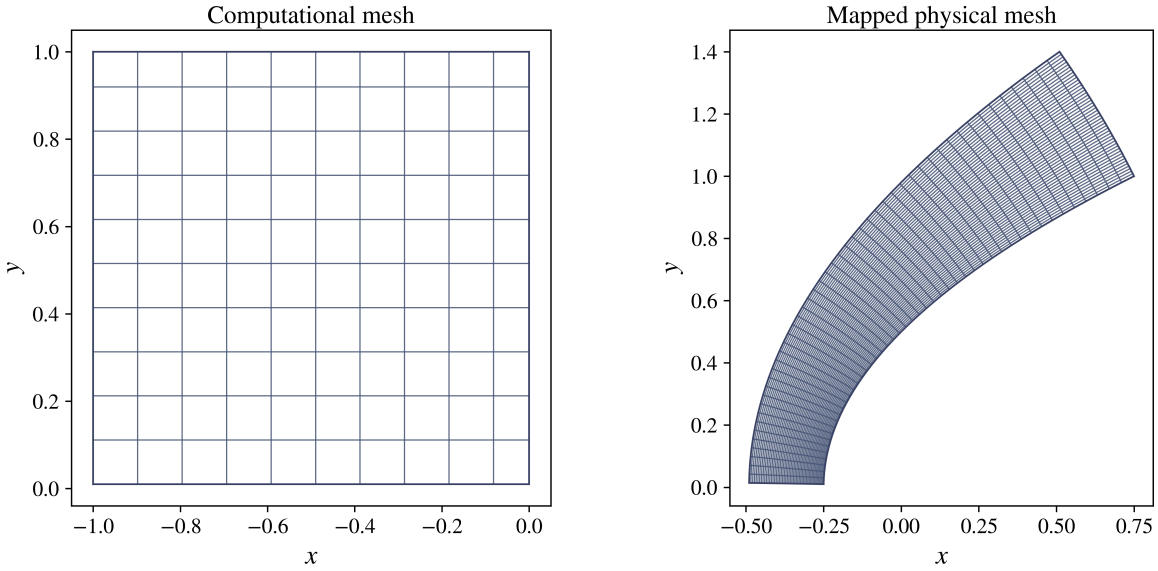
Expected outputs:

Expected output

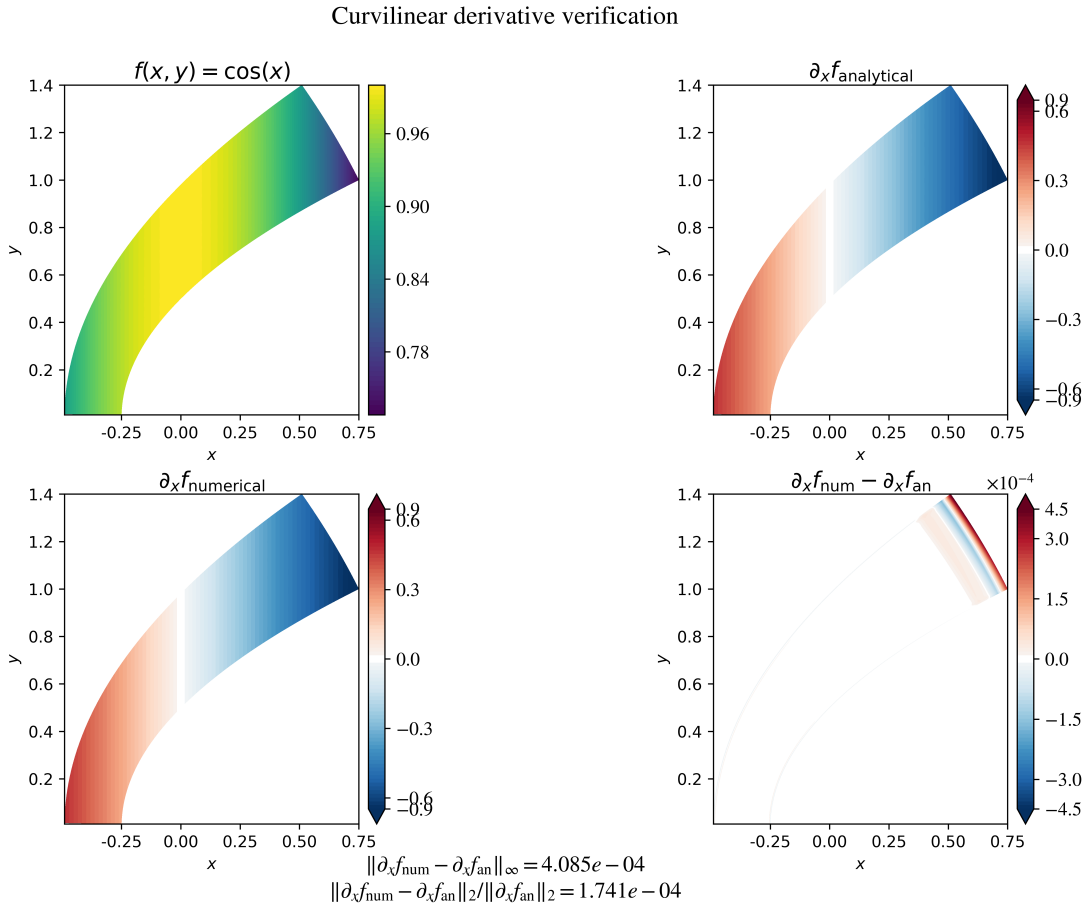
```
examples/example_0_curvilinear_validation.npz
examples/example_0_curvilinear_validation.png
examples/example_0_curvilinear_validation_baseflow.png
examples/example_0_curvilinear_validation_support_error.png
artifacts/curvilinear_derivative_parity.json
```

What to inspect: the mapped grid should remain smooth, and the support-error figure should localize the derivative error rather than showing a domain-wide failure. This case is mostly a geometry and derivative diagnostic.

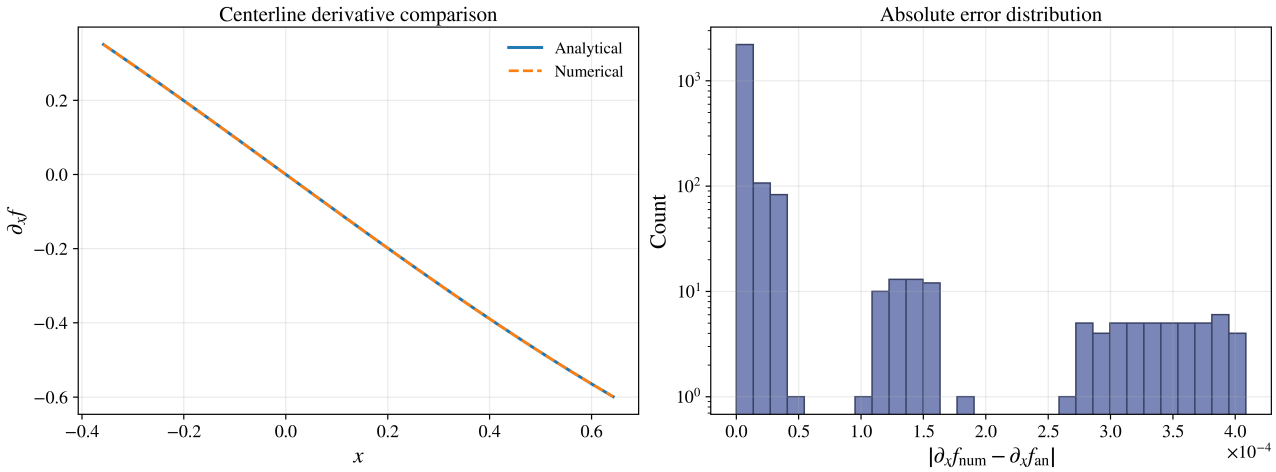
The derivative comparison uses `f = cos(x)`, so the analytic derivative is `-sin(x)`. The useful reading is not whether the plotted curves are visually smooth, but whether the error magnitude stays near the numerical tolerance and whether any larger error is confined to boundary or mapping-sensitive locations. If later figure work shows small oscillations near a boundary, document them as derivative-operator or closure behavior before treating them as a flow result.



Example 0 grid and mapped-domain view.



Example 0 derivative comparison.



Example 0 derivative-support error.

5.2 Example 1: Blasius resolvent

Example 1 is the first physical resolvent case. It studies planar perturbations of a spatially evolving compressible Blasius boundary layer at low Mach number. The Martini example uses this case to show Tollmien–Schlichting wave amplification in a Cartesian domain.

Start with the smoke case, then run the full native-resolution case:

Command

```
.venv/bin/python -m examples.example_1_blasius_smoke
.venv/bin/python -m examples.example_1_blasius
bash run_all_examples.sh --fresh --example 1 --skip-parity
```

Main knobs:

File excerpt

```
"Re": 6e5,
"Ma": 0.3,
"nx": 300,
"ny": 100,
"omega": 60.0,
"n_modes": 3,
"forcing_zero_vars": ["T", "rho", "w"]
```

The boundary condition is Dirichlet on all sides for `uvwT`:

File excerpt

```
"bc": {
  "penalty": 1e4,
  "recipe": [
    {"type": "dirichlet", "borders": "lrbt", "variables": "uvwT"}
  ]
}
```

Expected outputs:

Expected output

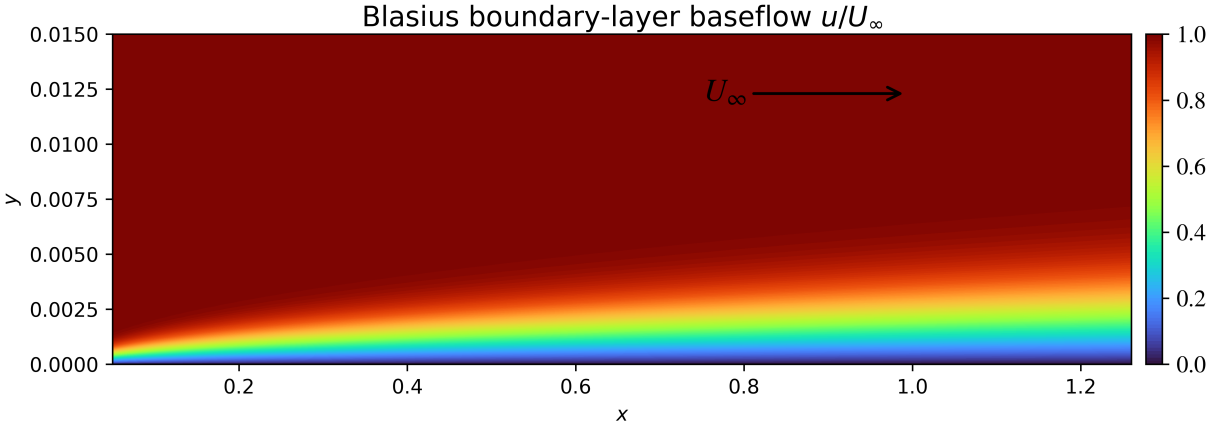
```
examples/example_1_blasius.npz
examples/example_1_blasius.png
examples/example_1_blasius_baseflow.png
examples/example_1_blasius_support_components.png
examples/example_1_blasius_support_components_shared_limits.png
```

What to inspect: the leading forcing is upstream, while the response is strongest downstream. This is the expected amplifier-flow signature of a non-normal boundary-layer operator. The leading response corresponds to the most efficiently forced Tollmien–Schlichting wavepacket. Suboptimal modes use similar physics but must be orthogonal to the leading mode, so their response patterns are modulated and their gains are lower.

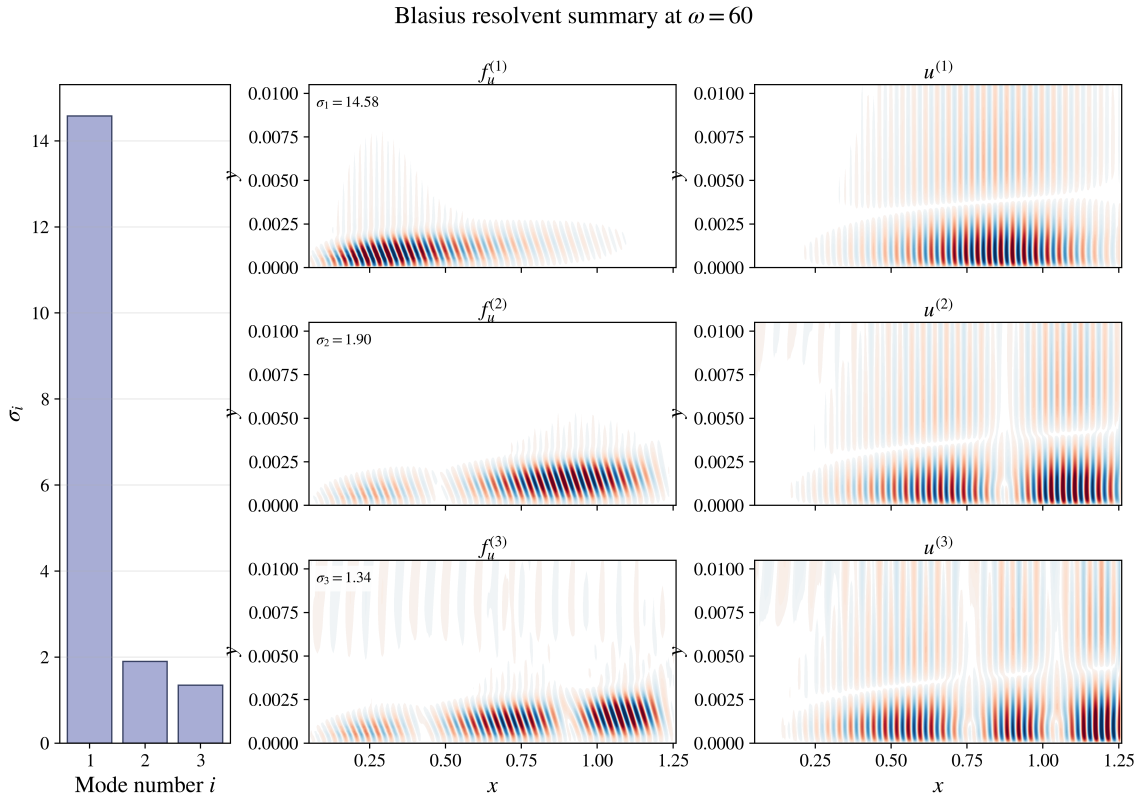
Use the baseflow figure to check the growing boundary layer and the downstream sponge before interpreting the modes. If the response or forcing support sits mainly inside the sponge, the plot is telling you about damping or boundary treatment, not the physical TS amplifier. The shared-limit support figure is the better diagnostic for where energy is concentrated across components; the individual-limit figure is better for seeing each component's shape.

Note

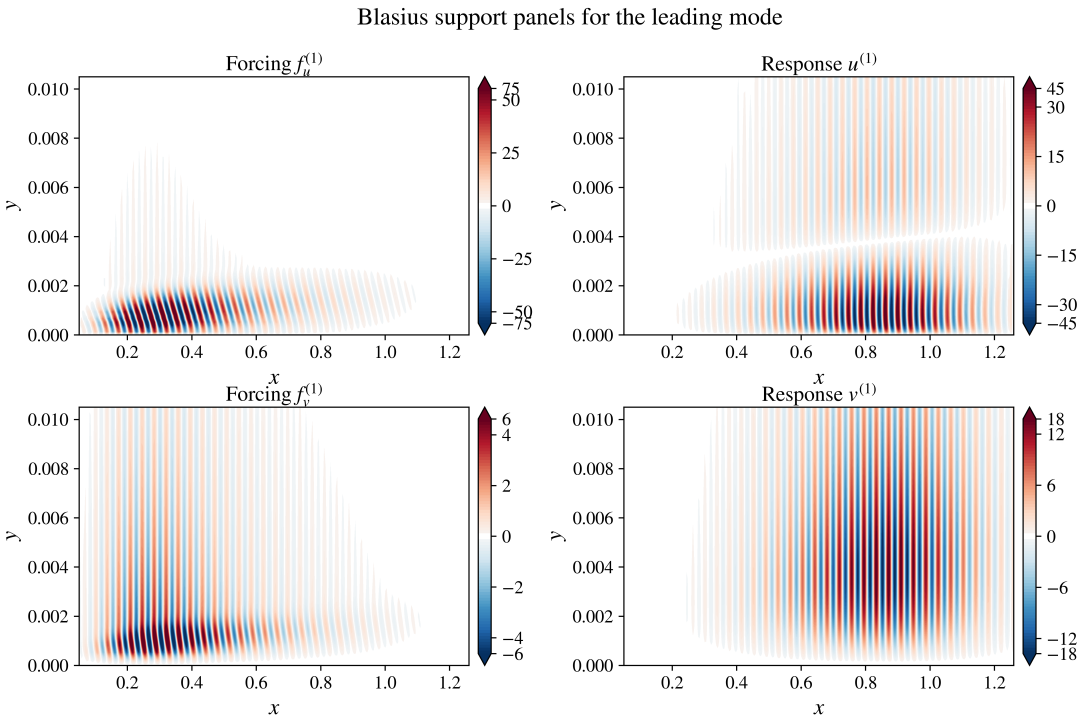
The verification note records `Re = 6e5` in MATLAB and Python. The paper text appears to understate this value as `60,000`; the MATLAB source and the Python port use `600,000`.



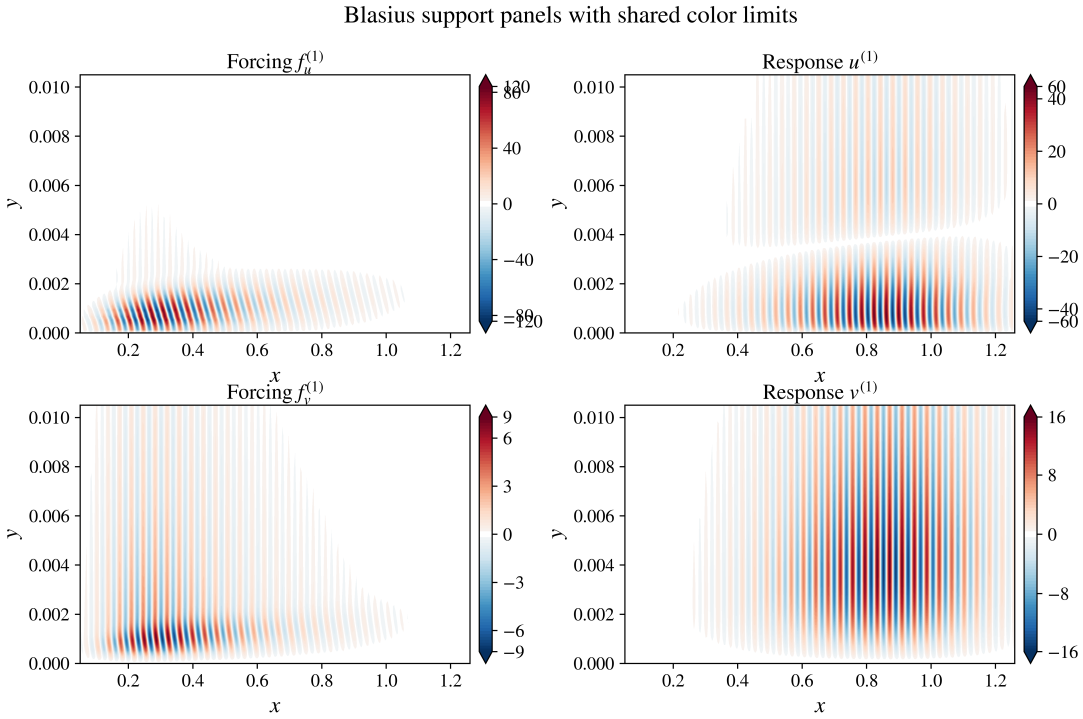
Example 1 Blasius base flow and sponge.



Example 1 leading resolvent gains, forcing modes, and response modes.



Example 1 forcing/response support components with individual color limits.



Example 1 support components with shared color limits.

5.3 Example 2: axisymmetric jet resolvent

Example 2 moves from Cartesian to cylindrical coordinates. It analyzes a Mach 0.9 turbulent jet mean flow and uses the axisymmetric branch with azimuthal mode `m = 0`. This case highlights Kelvin–Helmholtz wavepacket amplification in a spatially evolving jet.

Command

```
.venv/bin/python -m examples.example_2_jet
bash run_all_examples.sh --fresh --example 2 --skip-parity
```

Main knobs:

File excerpt

```
"St": 0.39,
"m": 0,
"model": "axy",
"coarsen": 1,
"fd_order": 4
```

Expected outputs:

Expected output

```
examples/example_2_jet.npz
examples/example_2_jet.png
examples/example_2_jet_baseflow.png
examples/example_2_jet_support_components.png
```

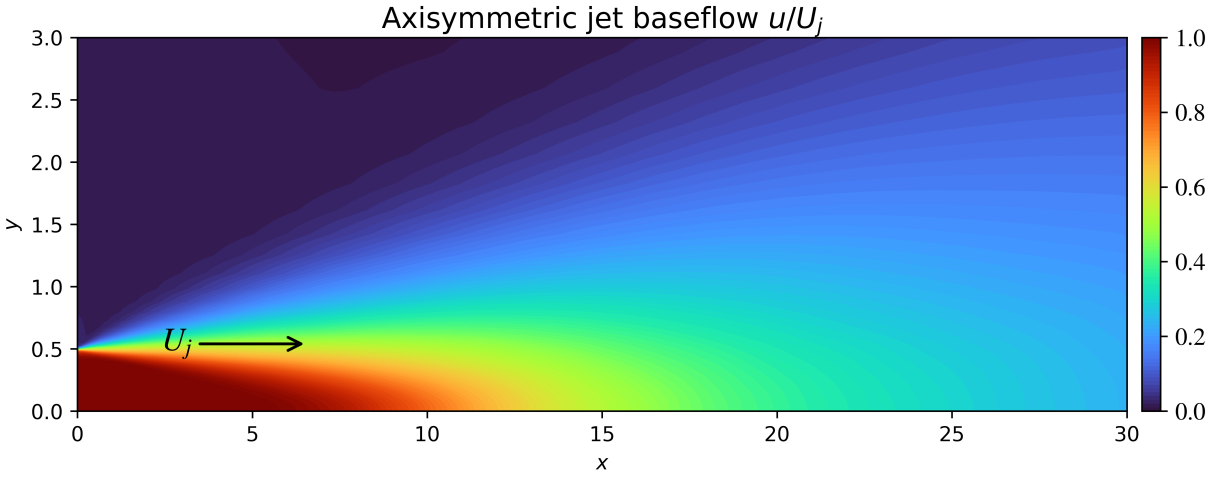
What to inspect: the leading forcing should be concentrated near the nozzle lip and mixing layer. The response forms a downstream wavepacket. The large leading gain is consistent with convective amplification associated with the inflectional jet mean flow. Higher modes have lower gain and more distributed support.

Here `St = 0.39` is converted in the script to `omega = 2*pi*St`, and `m = 0` means the perturbation is axisymmetric. In the figures, first locate the nozzle lip and the shear layer in the baseflow. Then check that forcing is near the receptive upstream mixing layer while response extends downstream as a wavepacket. A response that fills the far-field boundary or sponge would be a warning to inspect the domain and damping

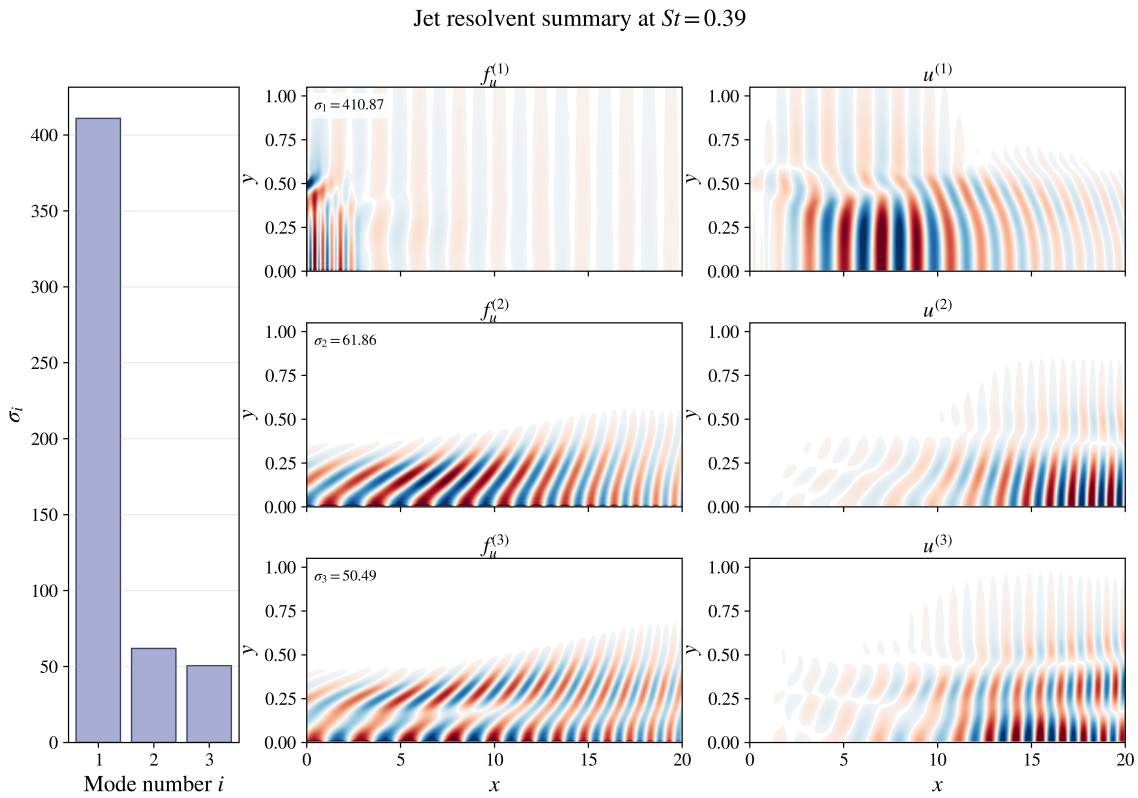
settings.

Note

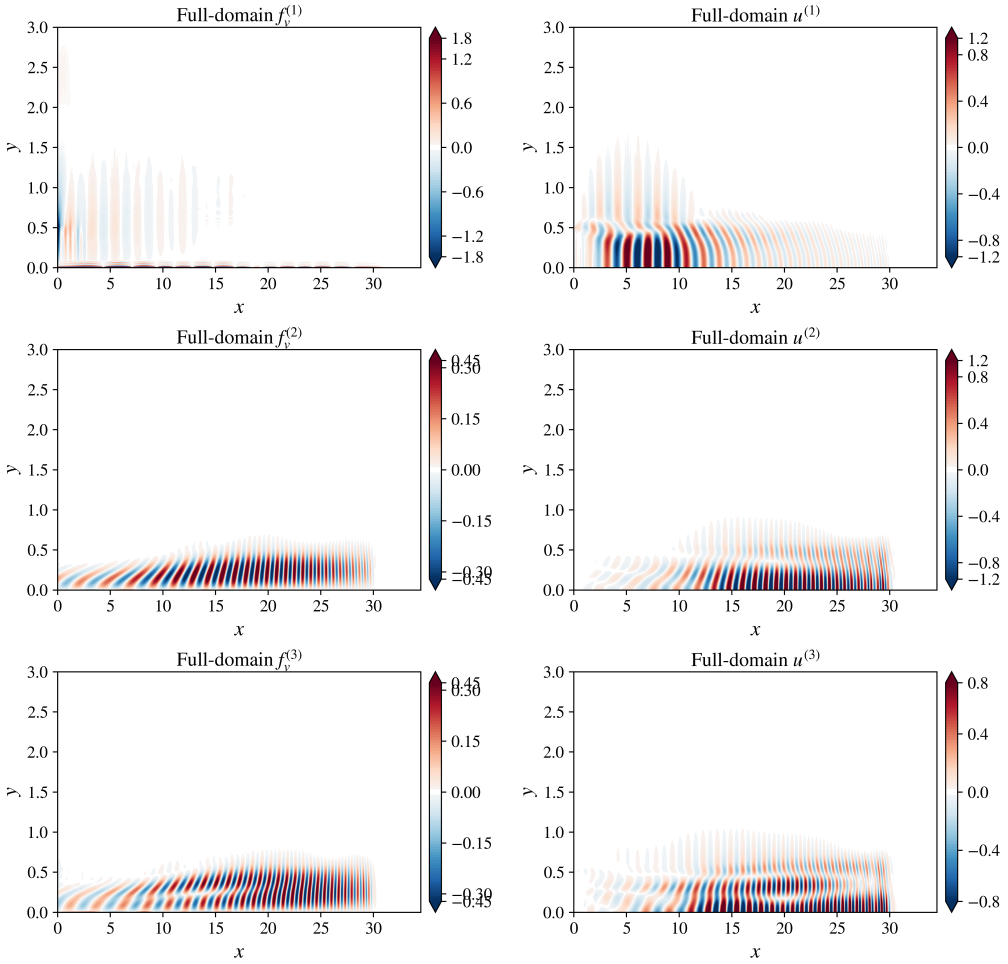
The jet base flow and sponge are loaded from the same `.mat` source used by the MATLAB example, so this case is a useful data-loading and axisymmetric-operator parity check.



Example 2 jet mean flow and coordinate system.



Example 2 axisymmetric jet resolvent modes and gains.



Example 2 forcing/response support components.

5.4 Example 3: streaky boundary-layer stability

Example 3 is a stability example rather than a resolvent example. It uses a locally parallel Mach 6 streaky boundary layer with periodicity in the spanwise direction and homogeneity in the streamwise direction. It demonstrates temporal eigenvalues, spatial eigenvalues, and Floquet handling of harmonic and subharmonic content.

Command

```
.venv/bin/python -m examples.example_3_streaky_bl
bash run_all_examples.sh --fresh --example 3 --skip-parity
```

Main knobs:

File excerpt

```
"nx": 125,
"ny": 250,
"x_periodicity": true,
"temporal_targets": [[0.59, 0.02], [0.626, -0.005]],
"temporal_quotas": [2, 8],
"n_modes": 50,
"ncv": 150,
"run_temporal": true,
"run_spatial": true
```

Expected outputs:

Expected output

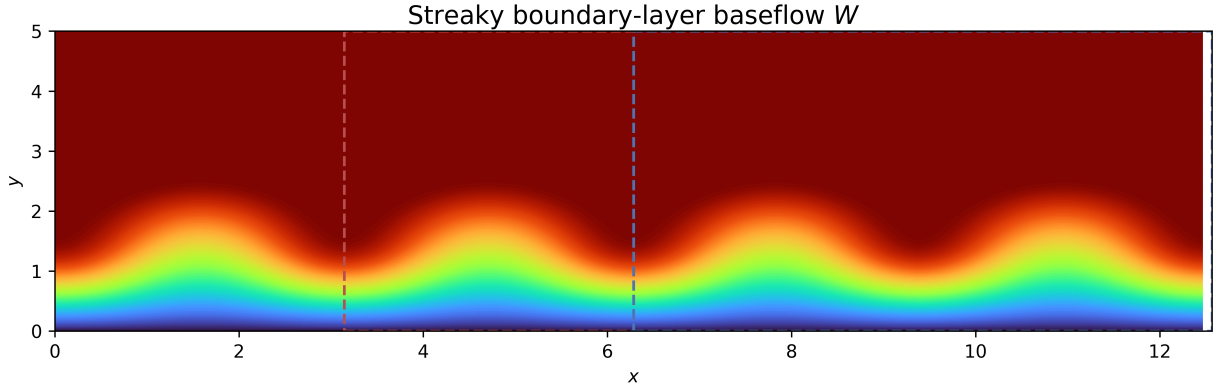
```
examples/example_3_streaky_bl.npz
examples/example_3_streaky_bl.png
examples/example_3_streaky_bl_baseflow.png
examples/example_3_streaky_bl_spatial.png
examples/example_3_streaky_bl_spectra.png
```

What to inspect: the temporal spectrum should identify unstable streak modes. The Floquet treatment lets the smaller domain separate harmonic and subharmonic branches. The spatial analysis follows the corresponding spatial branch and is used to reason about whether the instability is convective or absolute.

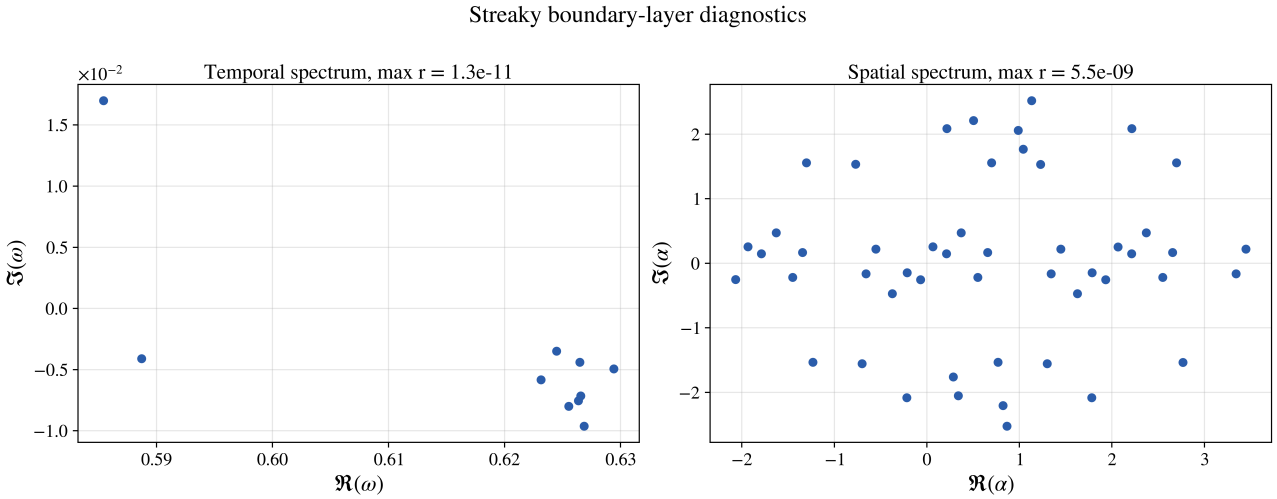
For the temporal spectrum, the horizontal coordinate is `Re(omega)` and the vertical coordinate is `Im(omega)`; positive `Im(omega)` indicates temporal growth under this convention. For the spatial plot, `alpha` is complex and the sign of spatial amplification depends on the branch being followed. Do not read convective or absolute behavior from one spatial point alone; first make sure the temporal spectrum is converged and the selected branch is the intended one.

Note

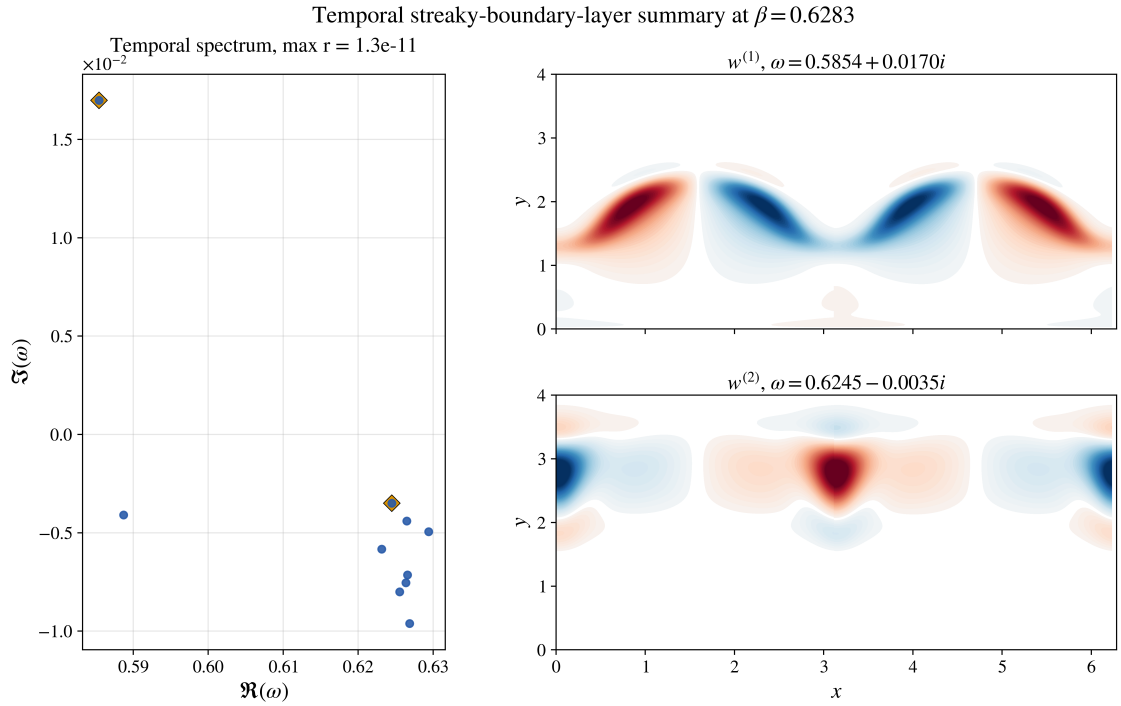
The verification note records MATLAB and Python using `125 x 250`. The paper text appears to report a lower grid count; the MATLAB source is the authoritative reference for the port.



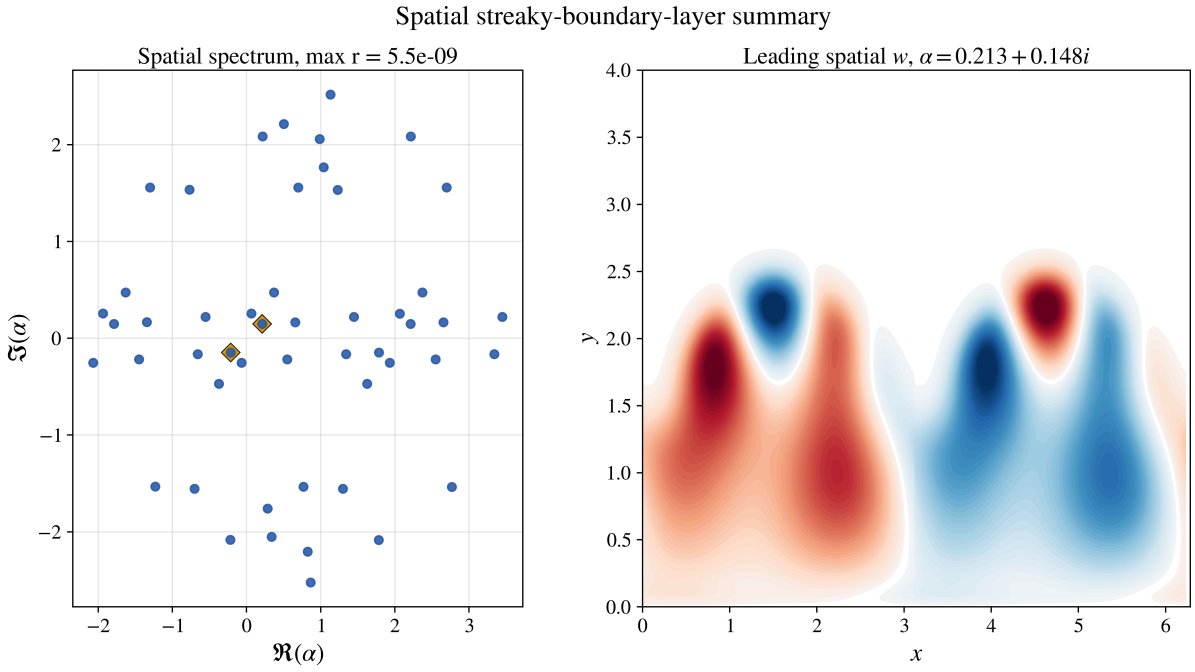
Example 3 streaky boundary-layer base flow.



Example 3 temporal and spatial spectra.



Example 3 temporal modes.



Example 3 spatial-stability branch.

Warning

This example is the most sensitive one for interpretation. If you are changing it, first converge the temporal spectrum before trusting spatial-branch plots. Increasing `ncv` or saved mode counts may be needed for detailed spectrum work, but that is a numerical-study change, not a tutorial change.

5.5 Example 4: cavity direct and adjoint EVP

Example 4 demonstrates a non-trivial geometry. The cavity is represented by a deformed grid and a mask, then direct and adjoint temporal eigenvalue problems are solved. Physically, the unstable modes are Rossiter-type feedback modes: shear-layer Kelvin–Helmholtz waves travel downstream, scatter near the downstream lip, and acoustic feedback inside the cavity can re-excite the shear layer.

Command

```
.venv/bin/python -m examples.example_4_cavity
bash run_all_examples.sh --fresh --example 4 --skip-parity
```

Main knobs:

File excerpt

```
"nx": 241,
"ny": 101,
"omega_target": [1.5, 1.0],
"n_modes": 50,
"ncv": 300,
"beta": 0.0,
"assembly_backend": "scipy"
```

Expected outputs:

Expected output

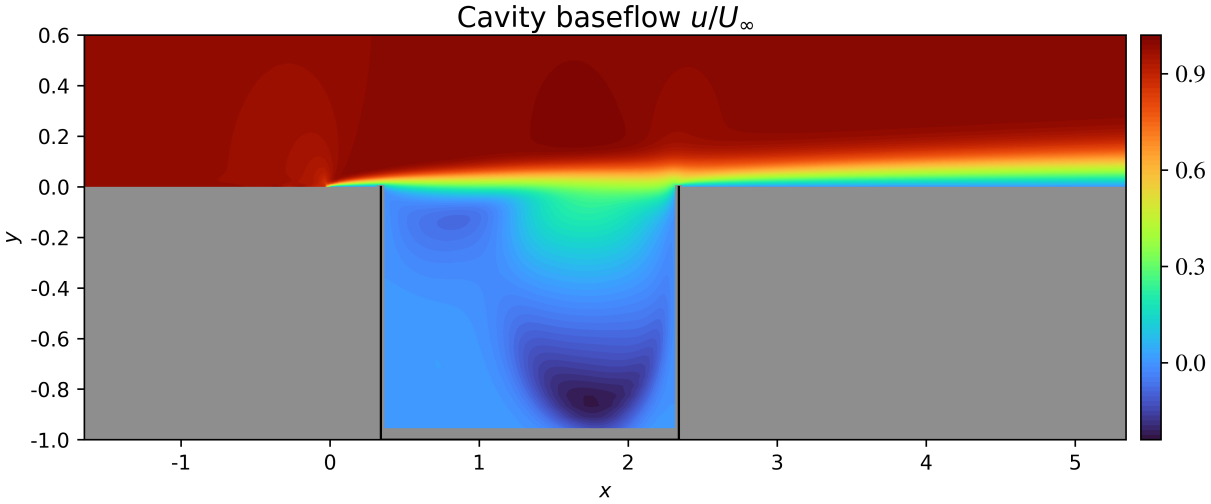
```
examples/example_4_cavity.npz
examples/example_4_cavity.png
examples/example_4_cavity_baseflow.png
examples/example_4_cavity_direct.png
examples/example_4_cavity_adjoint.png
```

What to inspect: the direct and adjoint spectra should overlay after the documented conjugation convention. The direct modes show the unstable cavity feedback structure. The adjoint support is strongest near the upstream cavity region, which marks where disturbances most efficiently excite the unstable direct modes.

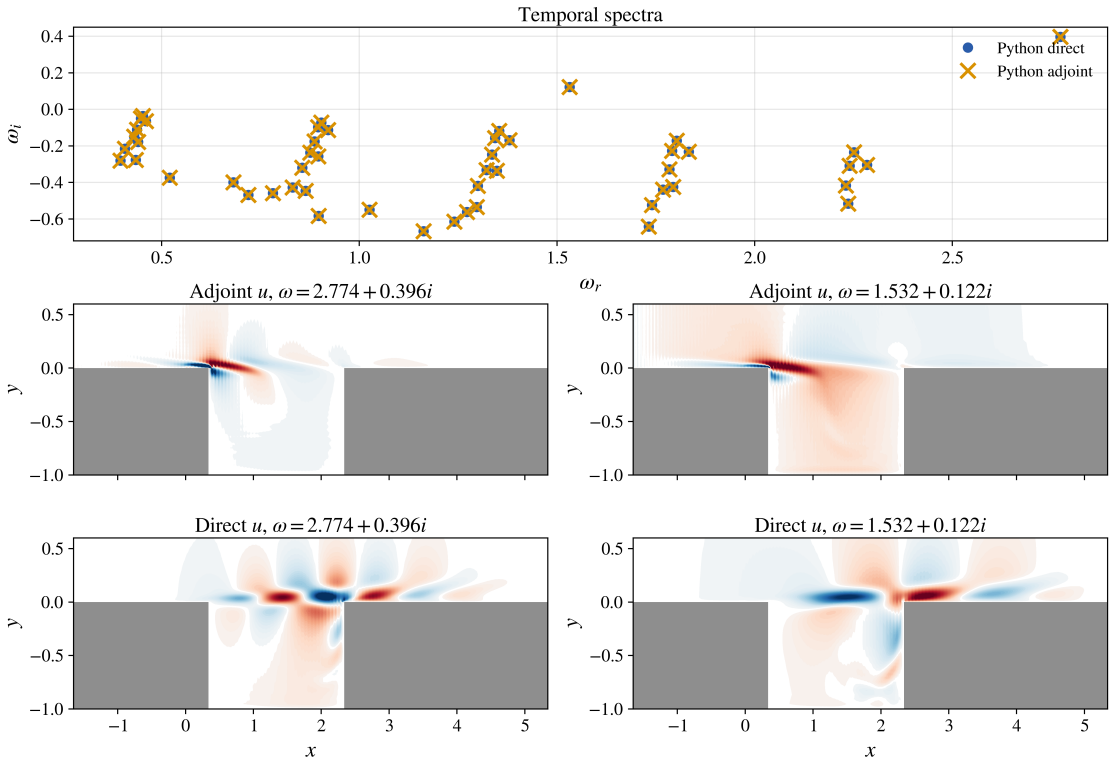
The selected direct and adjoint panels are diagnostics of the computed eigenpair set near the requested target, not a claim that a printed index is physically special by itself. If a label such as "mode 18" appears, interpret it as the retained eigenpair index in that run and check the spectrum location before assigning physical meaning. Fine oscillations in adjoint contours can also be a discrete-adjoint effect; the main quantity to read is the receptivity envelope and its overlap with the direct mode, not every small contour wiggle.

Note

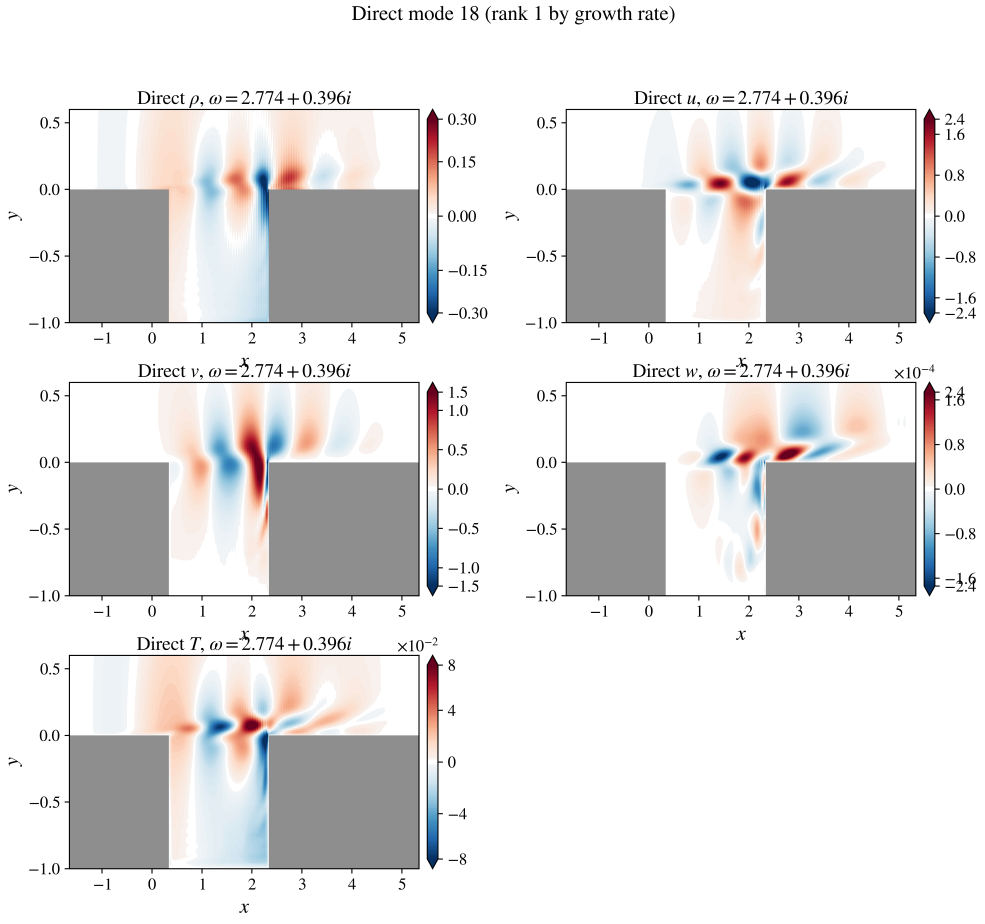
The cavity currently keeps `assembly_backend: "scipy"` because the adjoint workflow remains SciPy-based.



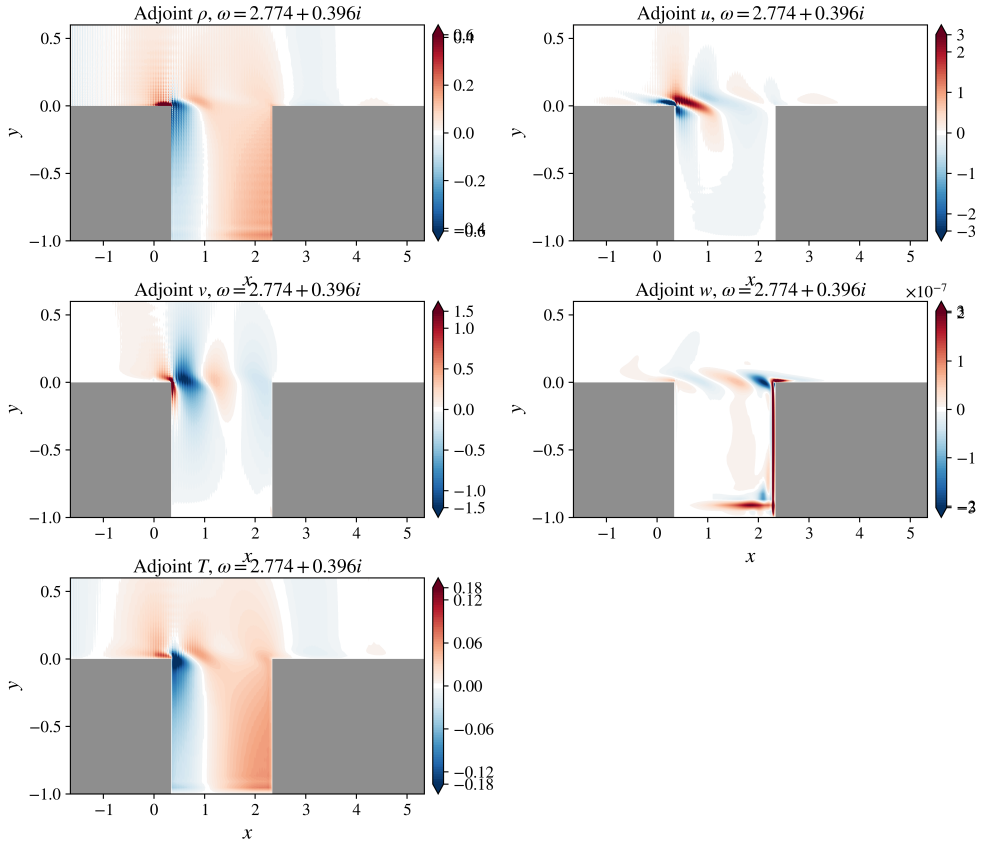
Example 4 deformed and masked cavity base flow.



Example 4 direct/adjoint spectra and selected mode support.



Example 4 selected direct cavity modes.



Example 4 selected adjoint cavity modes.

5.6 Example 5: paraboloid resolvent

Example 5 combines curvilinear coordinates with the axisymmetric model. It maps a Cartesian computational grid around a paraboloid body and computes resolvent modes for azimuthal mode `m = 5`. The example is intended to show receptivity and streak formation around a body rather than a simple rectangular flow.

Command

```
.venv/bin/python -m examples.example_5_paraboloid
bash run_all_examples.sh --fresh --example 5 --skip-parity
```

Main knobs:

File excerpt

```
"nx": 100,
"ny": 100,
"model": "axy",
"m": 5,
"omega": 0.0,
"use_omega_shift_fallback": true,
"forcing_zero_vars": ["T", "rho"]
```

Expected outputs:

Expected output

```
examples/example_5_paraboloid.npz
examples/example_5_paraboloid.png
examples/example_5_paraboloid_baseflow.png
```

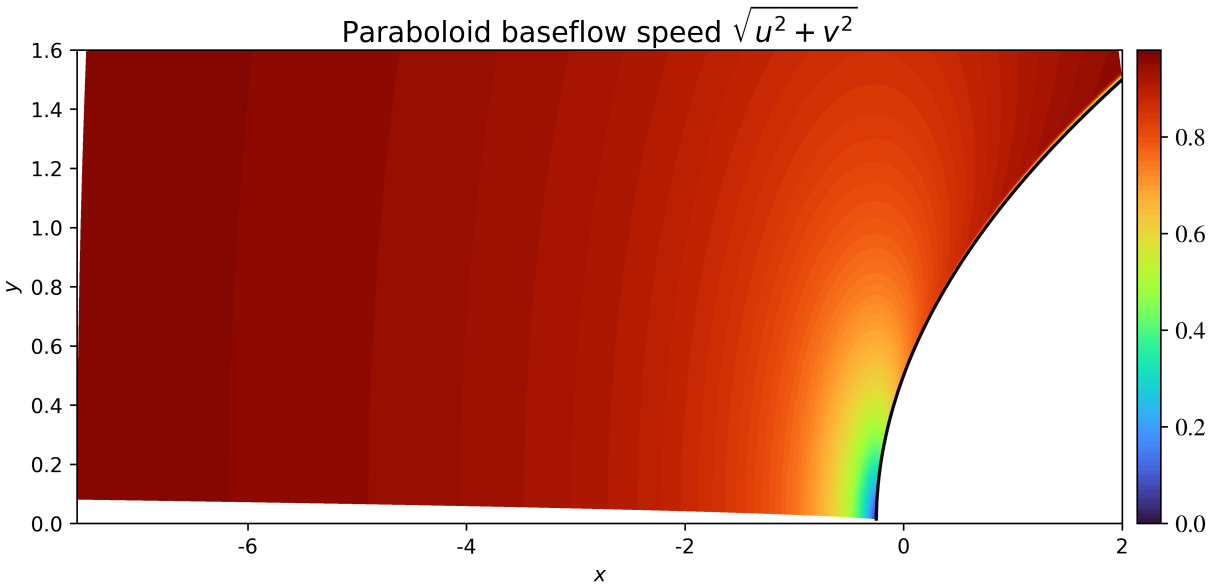
```
examples/example_5_paraboloid_support_components.png
```

What to inspect: forcing and response concentrate around the body and stagnation-region boundary layer. The mechanism is tied to streamwise vortices that enter the boundary layer and create downwash/upwash patterns; these redistribute high- and low-speed fluid and form streaks through lift-up. Suboptimal modes show additional radial or wall-normal nodes and lower gains.

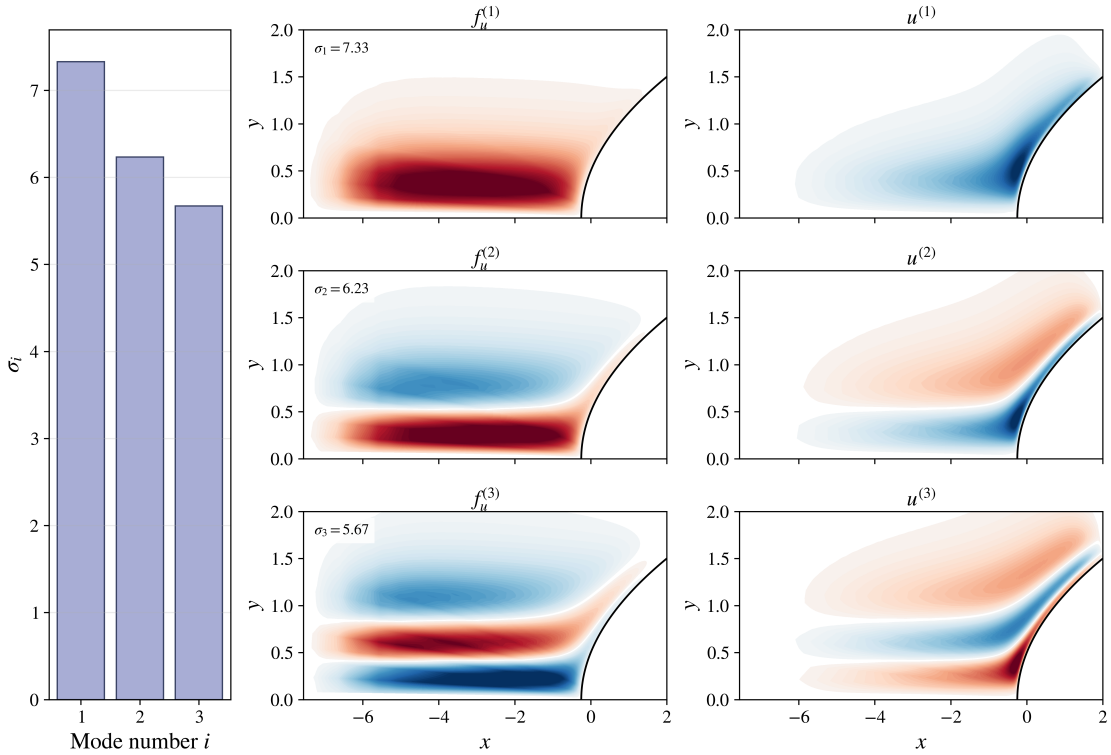
This is a nominal zero-frequency resolvent case. The script can use a small imaginary frequency shift when the zero-frequency operator is singular or ill-conditioned; the cache records the frequency actually used. In the figures, blank or clipped regions can come from the mapped physical domain, body geometry, or plotting limits, so inspect the baseflow/body outline before concluding that the baseflow itself is missing data.

Note

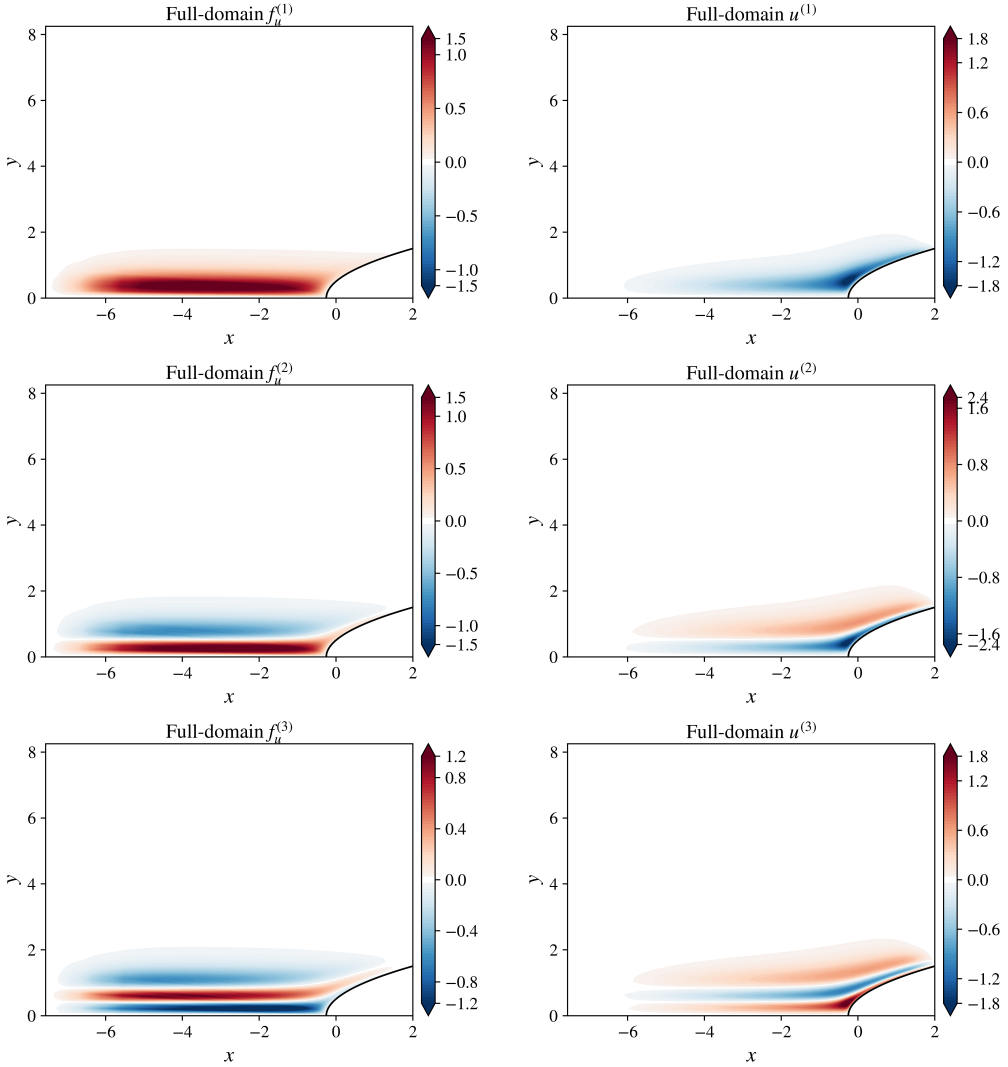
The Python port includes the physical-domain scaling before the conformal mapping. The earlier missing-scaling issue recorded in the verification note is not a current tutorial warning.



Example 5 curvilinear paraboloid base flow.



Example 5 paraboloid resolvent modes and gains.



Example 5 forcing/response support components.

5.7 Figure-quality notes for the next figure pass

The current figures are usable for this tutorial, but several can be improved for publication-style reporting. The most common improvements are LaTeX-rendered axis labels, closer title placement, tighter colorbar spacing, scientific notation for small values, and shared color limits for support-component comparisons. Example-specific items worth addressing in the plotting code are:

- Example 0: show the full physical grid more explicitly, close the physical-domain outline, reduce column spacing, and use scientific notation for small derivative errors.
- Examples 1, 2, and 5: reduce spacing between mode panels, move titles closer to the plotted area, keep freestream arrows horizontal where shown, and tighten colorbar gaps.
- Example 3: verify the baseflow panel uses the intended streamwise component `W`, move spectrum legends away from eigenvalues, converge the temporal spectrum before using the spatial branch for claims, and save enough modes to identify the second unstable branch when that study is run.
- Example 4: review cavity-baseflow masking and extent against the MATLAB figure, keep spectrum-axis labels from overlapping neighboring panels, and document the selected direct/adjoint mode indices in the plotting caption or metadata.
- All examples: use consistent math fonts, colorbar tick spacing, larger axis labels where panels are dense, and white-centered diverging colormaps for signed modes.

5.8 Martini parity notes

The Python examples are also parity checks against the Martini MATLAB source. The comparison script checks saved gains, spectra, and extracted artifacts where reference bundles are available. Use these notes when a result appears to disagree with the paper text:

Expected output

```
Example 1  gains and mode artifacts   paper Re text appears inconsistent
Example 2  gains, baseflow, sponge    .mat source matches MATLAB path
Example 3  temporal/spatial spectra   MATLAB/Python grid is 125 x 250
Example 4  direct/adjoint spectra     adjoint convention is post-processed
Example 5  gains and mapped domain    mapping scaling is fixed in Python
```

6. Run the Martini parity checks

After the example artifacts exist, compare Python outputs with extracted MATLAB bundles:

Command

```
.venv/bin/python validation/martini2024/tools/compare_martini_examples.py
```

The machine-readable registry is:

Expected output

```
validation/martini2024/tools/martini_cases.jsonc
```

The current reports are written under:

Expected output

```
validation/martini2024/reports/martini_parity_latest.json
validation/martini2024/reports/martini_parity_table.csv
```

Warning

If the MATLAB NPZ bundles are absent, the comparison reports missing-bundle failures. That is a data-availability result, not a reason to loosen tolerances or edit tests.

For a complete example pass, use the bundled runner:

Command

```
bash run_all_examples.sh
```

7. Song 2023 validation baseline

The Song 2023 case is a validation workflow for an oblique shock / boundary-layer interaction. It is larger than the Martini examples and should be treated as a study, not as a quick smoke test.

The main files are:

Expected output

```
validation/song_2023/song_2023.py
validation/song_2023/configs/song_2023.jsonc
validation/song_2023/song_2023.sh
```

Run the baseline through the wrapper:

Command

```
bash validation/song_2023/song_2023.sh
```

The baseline JSONC contains the physical parameters, finite-difference order, stride, preprocessing, boundary conditions, and solver settings.

File excerpt

```
"Re": 1e5,
"Ma": 2.15,
"beta": 19.2,
"fd_order": 2,
"stride_ksi": 1,
"stride_eta": 1,
"run_stability": true,
"run_resolvent": false
```

Boundary conditions:

File excerpt

```
{"type": "dirichlet", "borders": "lt", "variables": "ruvwT"}
{"type": "dirichlet", "borders": "b",  "variables": "ruvw"}
{"type": "neumann",   "borders": "b",  "variables": "T"}
{"type": "neumann",   "borders": "r",  "variables": "ruvwT"}
```

Expected canonical outputs:

Expected output

```
validation/song_2023/song_2023.npz
validation/song_2023/song_2023.log
validation/song_2023/song_2023.png
validation/song_2023/song_2023_baseflow.png
validation/song_2023/song_2023_support_direct_modes.png
```

8. Song validation matrices

Use JSONC matrix files when you want to compare solver choices, preprocessing choices, or finite-difference orders without overwriting the baseline case.

Expected output

```
validation/song_2023/configs/song_2023_matrix.jsonc
validation/song_2023/configs/order_convergence.jsonc
validation/song_2023/configs/song_2023_order_preflight.jsonc
validation/song_2023/configs/song_2023_scaling.jsonc
validation/song_2023/configs/fd4_sensitivity.jsonc
validation/song_2023/configs/shift_scan.jsonc
validation/song_2023/configs/shifted_ksp_matrix.jsonc
```

Run the solver/preprocessing matrix:

Command

```
.venv/bin/python -m validation.song_2023.scripts.run_song_2023_matrix
```

Run the finite-difference order preflight before the full order convergence study:

Command

```
.venv/bin/python -m validation.song_2023.scripts.run_song_2023_order_preflight
```

Run the order convergence study only on a machine with enough memory for the configured guard:

Command

```
.venv/bin/python -m validation.song_2023.scripts.run_song_2023_order_convergence
.venv/bin/python validation/song_2023/scripts/plot_order_convergence.py
```

Warning

The full Song order-convergence study can require many gigabytes of memory. Check `memory_guard` in `validation/song_2023/configs/order_convergence.jsonc` before running it.

9. How to adjust a shipped case

For small experiments, copy the closest Python/JSONC pair into a working location. This keeps the shipped examples intact.

Command

```
mkdir -p tutorial/work
cp examples/example_1_blasius.py tutorial/work/my_blasius.py
cp examples/example_1_blasius_smoke.jsonc tutorial/work/my_blasius.jsonc
PYTHONPATH=src:. .venv/bin/python tutorial/work/my_blasius.py
```

Then edit:

Expected output

```
tutorial/work/my_blasius.jsonc
```

Good first changes:

- reduce `nx` and `ny` for fast tests,
- change `n_modes` only after the first successful run,
- change `fd_order` one value at a time,
- keep `assembly_backend: "scipy"` until the case works,
- keep solver settings in JSONC.

Note

The copied Python file uses `SCRIPT.with_suffix(".jsonc")`. That is why `my_blasius.py` reads `my_blasius.jsonc` automatically when the names match.

10. How to start a new case

Use this checklist before writing new package code:

- choose the closest existing case: Blasius for 2-D resolvent, jet/paraboloid for axisymmetric resolvent, streaky/cavity for EVP branches, Song for SWBLI validation;
- copy the Python/JSONC pair into a study directory;
- keep all physical and numerical parameters in the JSONC file;
- load base-flow data through a small loader rather than hardcoded absolute paths;
- define the boundary-condition recipe explicitly;
- run a reduced grid first;
- save `.npz` outputs every run and plot from the saved file;
- add a test or parity check before changing shared operator code.

For MATLAB-backed parity, add a registry entry only after the reference bundle location is known:

Expected output

```
validation/martini2024/tools/martini_cases.jsonc
```

11. Common first-run problems

- If `petsc4py` or `slepc4py` cannot import, rerun `bash install.sh -check`.
- If a run appears to use the wrong Python, use `.venv/bin/python` explicitly.
- If a full example is too slow, run the smoke or reduced-grid variant first.
- If a parity report says a MATLAB NPZ bundle is missing, generate or provide the bundle; do not change tolerances to hide the missing input.

- If a high-order run changes behavior, compare `fd_order` values through a JSONC matrix instead of editing operator code first.

12. Build this tutorial

The tutorial source and Makefile live together:

Expected output

```
tutorial/tutorial.tex
tutorial/Makefile
```

Build from inside the tutorial directory:

Command

```
cd tutorial
make
```

The build writes:

Expected output

```
tutorial/tutorial.pdf
```